

# Computer Graphics (CSC214)

## Unit - 10: Introduction to OpenGL

Compiled by Shishir Ghimire

Credit Hours: 3hrs

# ► TABLE OF CONTENTS

## Unit 10: Introduction to OpenGL (3 Hrs.)

1.1 Introduction, Callback functions, Color commands, Drawings pixels, lines, polygons using OpenGL, Viewing and Lighting

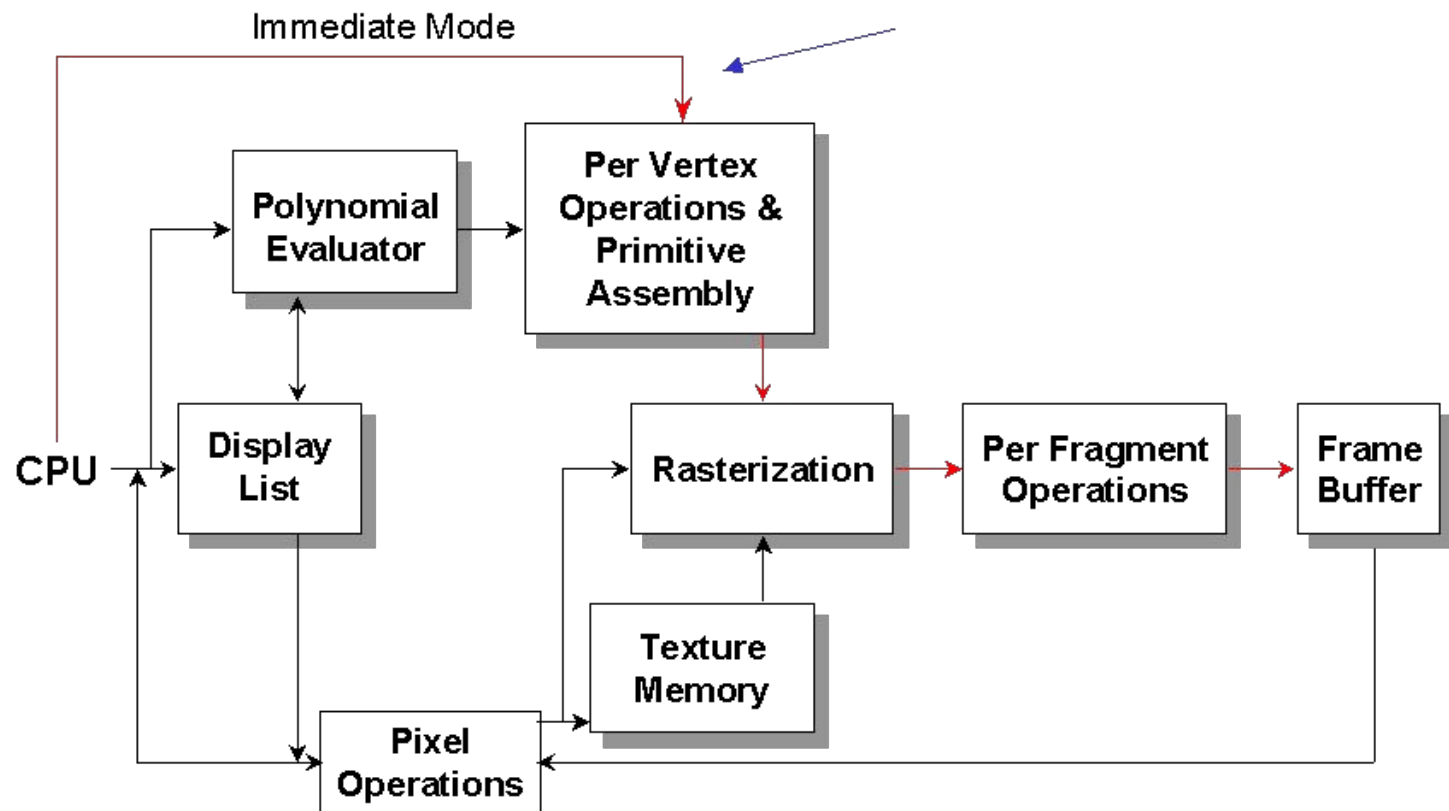


# ► Introduction to OpenGL

## ► OPENGGL:

- ❖ OpenGL is a **software interface** that allows a programmer to **communicate** with graphics hardware.
- ❖ OpenGL is the short form for "**Open Graphics Library**."
- ❖ It is an **application programming interface (API)** designed for rendering 2D and 3D graphics.
- ❖ It provides a **common set of commands** that can be used to manage graphics in different applications and on multiple platforms.
- ❖ **Examples** of OpenGL commands include drawing polygons, assigning colors to shapes, zooming in and out, rotating objects, etc.
- ❖ The **OpenGL Utility Toolkit (GLUT)** has been created to aid in the development of more complicated 3D objects such as a **sphere, a torus, and even a teapot.**
- ❖ Like all software libraries, OpenGL consists of a **series of function calls that can be invoked** from our own programs.

## ► OpenGL Architecture:



## ► A History of OpenGL:

- ❖ Originated from SGI's **Iris GL** - OpenGL
- ❖ "Open" standard allowing for a wide range of hardware platforms.
  - OpenGL v1.0 (1992)
  - OpenGL v1.1 (1995)
  - OpenGL v1.4 (2002)
  - OpenGL v4.6 (2017) - **Latest**
- ❖ Governed by the **OpenGL Architecture Review Board (ARB)**

**Note:** "Mesa" - an open-source implementation project of OpenGL, Vulkan etc  
(<https://mesa3d.org/>)

Visit this website to learn **OpenGL:** <https://learnopengl.com/Getting-started/OpenGL>

## ► Why OpenGL?

- ❖ The **first vendor-independent API** for development of graphics applications.
- ❖ There is **no need to license** it to use it.
- ❖ It was designed to **use with the graphics card** where possible to improve performance.
- ❖ **Originally based on a state machine**, procedural model, thus it can be used with a wide variety of programming languages.

## ► Access to Libraries:

### ❖ For Windows System:

- The header inclusion for windows system will include:

```
#include <windows.h>
```

```
#include <GL/glut.h>
```

### ❖ For Linux System:

- The header inclusion for linux system will include:

```
#include <X11/Xlib.h>
```

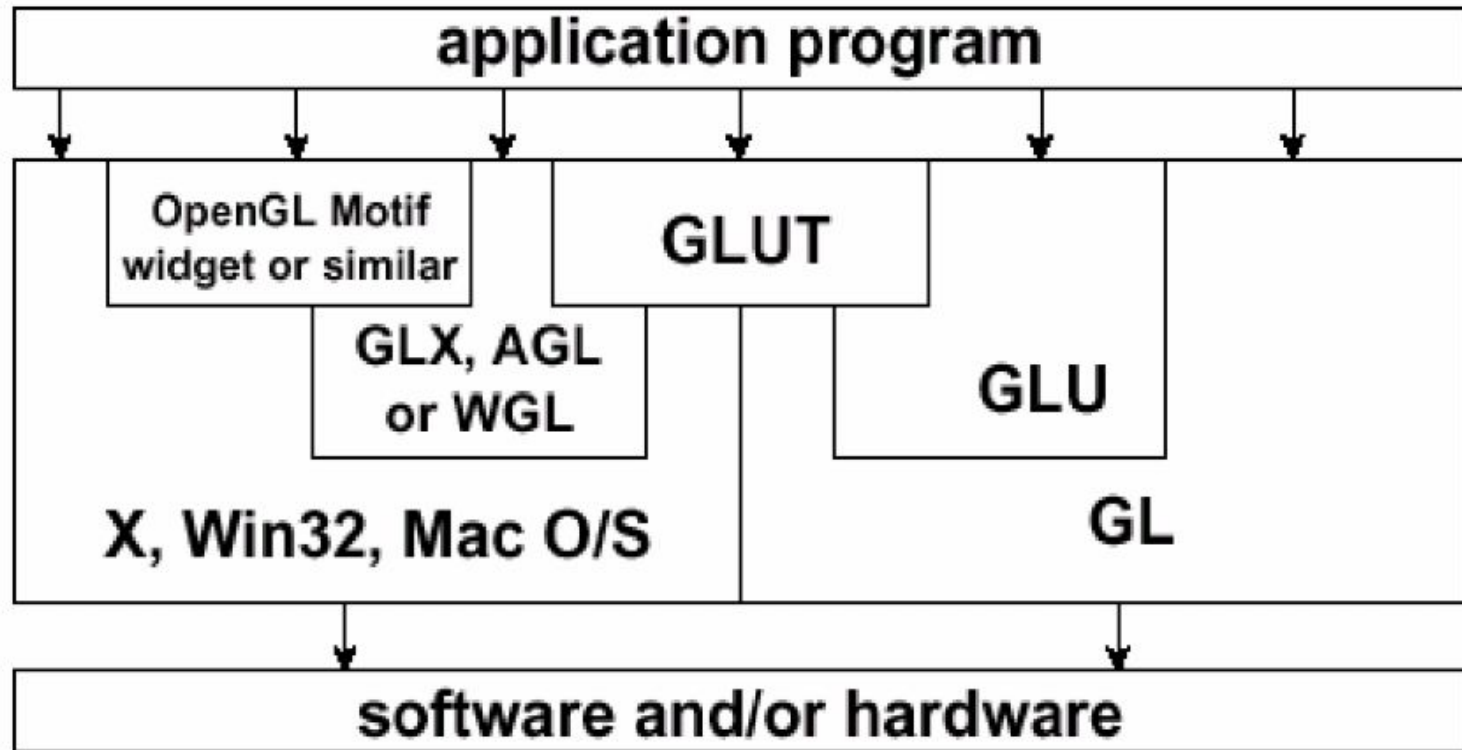
```
#include <GL/glut.h>
```



## ► OpenGL Data Types:

| <b><i>OpenGL Data Type</i></b> | <b><i>Internal Representation</i></b> | <b><i>Defined as C Type</i></b> | <b><i>C Literal Suffix</i></b> |
|--------------------------------|---------------------------------------|---------------------------------|--------------------------------|
| GLbyte                         | 8-bit integer                         | Signed char                     | b                              |
| GLshort                        | 16-bit integer                        | Short                           | s                              |
| GLint, GLsizei                 | 32-bit integer                        | Long                            | l                              |
| GLfloat, GLclampf              | 32-bit floating point                 | Float                           | f                              |
| GLdouble, GLclampd             | 64-bit floating point                 | Double                          | d                              |
| GLubyte, GLboolean             | 8-bit unsigned integer                | Unsigned char                   | ub                             |
| GLushort                       | 16-bit unsigned integer               | Unsigned short                  | us                             |
| GLuint, GLenum, GLbitfield     | 32-bit unsigned integer               | Unsigned long                   | ui                             |

## ► OpenGL API Hierarchy :



## ► OpenGL Division of Labor:

### ❖ GL:

- The "core" library of OpenGL that is **platform independent**.

### ❖ GLU:

- This directory holds the **GLU library files**, which offer useful functions for **simplifying common tasks**, such as creating quadrics (ellipses, parabolas, and hyperbolas), handling textures, and performing various geometric transformations.

### ❖ GLUT/AUX:

- The GLUT library files reside here, allowing developers to **create and manage OpenGL windows**, handle events (like mouse clicks and keyboard presses), and manage the rendering process.

## ► Libraries and Headers:

| Category          | Library File (PC)     | Library File (UNIX) | Header File     | Notes                                    |
|-------------------|-----------------------|---------------------|-----------------|------------------------------------------|
| OpenGL            | opengl32.lib          | -lgl                | gl.h            | "core" library                           |
| Auxiliary library | glu32.lib             | -lglu               | glu.h           | handles a variety of accessory functions |
| Utility toolkits  | glut32.lib, glaux.lib | -lglut, -lglaux     | glut.h, glaux.h | window management                        |

All are presented in the C language.

## ► Headers:

### ❖ GL/gl.h

- This header file provides the core **OpenGL definitions and function prototypes**. It contains the fundamental functions for rendering geometric primitives and managing graphical states.

### ❖ GL/glu.h

- The GLU header file defines functions for simplifying common tasks, such as **creating quadrics, managing textures**, and performing various geometric transformations.

### ❖ GL/glut.h

- The GLUT header file provides functions for creating and managing OpenGL windows, **handling input events, and managing the rendering process**. It allows developers to easily set up an OpenGL context and interact with the user.

## ► Environment Setup:

- ❖ **Language:** Discussions will be in C/C++.
- ❖ **Library:** Use the GLUT library for window management.
- ❖ **Necessary Files:**
  - `gl.h`
  - `glu.h`
  - `glut.h`
  - `opengl32.lib`
  - `glu32.lib`
  - `glut32.lib`
- ❖ **Instructions:**
  - Download the necessary files from ***<http://www.opengl.org>***.

## ► How to Use:

- ❖ Include the necessary header files in your code:
- ❖ Only the "core" library (*opengl32.lib, gl.h*) is required.

```
#include <GL/gl.h>    // "core", the only thing that is required
#include <GL/glu.h>    // handles accessory functions
#include <GL/glut.h>   // handles window management

void main( int argc, char **argv )
{
    . . . . .
}
```

## ► OpenGL Function Naming:

`glColor3f(...)`

gl library    Root command, # of arguments, type of arguments

gl means OpenGL

glu means GLU

glut means GLUT

f: the argument is *float* type

i: the argument is *integer* type

v: the argument requires a *vector*



## ► **Basic OpenGL Coding Framework:**

### **1. Configure GL (and GLUT)**

- Open window, Display mode, etc.

### **2. Initialize OpenGL state**

- Background color, light, view positions, etc.

### **3. Register callback functions**

- Render, Interaction (keyboard, mouse), etc.

### **4. Event processing loop**

- `glutMainLoop()`

## ► Sample Program:

1. Configure GL (and GLUT)
2. Initialize OpenGL state
3. Register callback functions
4. Event processing loop

```
void main (int argc, char **argv)
{
    // Step 1: Configure GL (and GLUT)
    glutInit (&argc, argv);
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize (500, 500);
    glutCreateWindow ("My First Program");

    // Step 2: Initialize OpenGL state
    myinit ();

    // Step 3: Register callback functions
    glutDisplayFunc (display);
    glutReshapeFunc (resize);
    glutKeyboardFunc (key);

    // Step 4: Enter the event processing loop
    glutMainLoop ();
}
```

## ► Callback Functions:

- ❖ A callback function is a function which the **library (GLUT) calls** when it **needs to know** how to process something.
- ❖ Eg. When GLUT gets a **key down event** it uses the **glutKeyboardFunc** **callback routine** to find out what to do with a key press.
- ❖ The following are some valid callback functions:

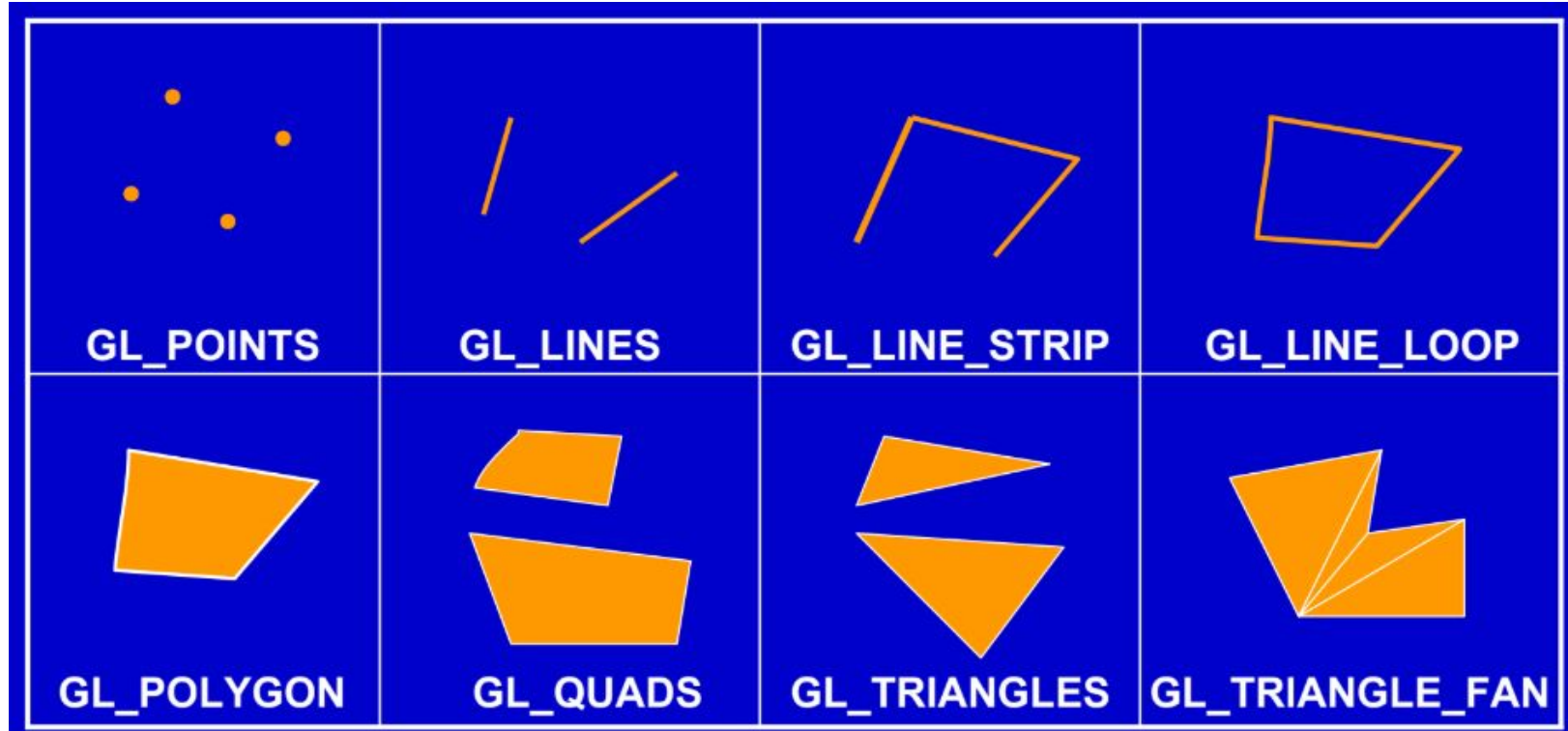
## Callback Functions:

| Callback                       | Description                                                                                                                                                 |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>GLU_TESS_BEGIN</b>          | The GLU_TESS_BEGIN callback is invoked like glBegin to indicate the start of a (triangle) primitive. The function takes a single argument of type GLenum.   |
| <b>GLU_TESS_BEGIN_DATA</b>     | The GLU_TESS_BEGIN_DATA is the same as GLU_TESS_BEGIN callback, except that it takes an additional pointer argument.                                        |
| <b>GLU_TESS_EDGE_FLAG</b>      | The GLU_TESS_EDGE_FLAG callback is similar to glEdgeFlag. This function takes a single boolean flag that indicates which edges lie on the polygon boundary. |
| <b>GLU_TESS_EDGE_FLAG_DATA</b> | The GLU_TESS_EDGE_FLAG_DATA is the same as GLU_TESS_EDGE_FLAG callback, except that it takes an additional pointer argument.                                |
| <b>GLU_TESS_END</b>            | The GLU_TESS_END callback indicates the end of a primitive, and it takes no arguments.                                                                      |

## ► Color Commands:

- ❖ OpenGL has two color models; **the RGBA mode** and **color index mode**.
- ❖ **In RGBA mode**, a color is specified by **three intensities** (for the **Red**, **Green**, and **Blue** components of the color) and optionally a fourth value, **Alpha**, which controls **transparency**.
- ❖ The function **`glColor4f(red, green, blue, alpha)`** maps available red, green, and blue intensities onto **[0.0 - 1.0]**, where **0.0 means** that the color component is **absent** (0.0, 0.0, 0.0) indicating black, and **1.0 is a saturated color** (1.0, 1.0, 1.0) indicating white.
- ❖ The **number of bits** used to represent each color **depends on the graphics card**. Current graphics cards have several megabytes of memory and use **24 or 32 bits for color**. The term "**bit plane**" refers to an image of **single-bit values**. Thus, a system with 24 bits of color has 24 bit planes.

## ► Geometric Primitives:



## ► Geometry Commands:

### 1. **glBegin(GLenum type)**

- Marks the **beginning of a vertex-data list** that describes a geometric primitive.

### 2. **glEnd(void)**

- Marks the **end of a vertex-data** list.

### 3. **glVertex\*(...)**

- **Specifies a vertex** for describing a geometric object.

## ► Specifying Geometric Primitives:

```
glBegin(type);  
    glVertex*(...);  
    .....  
    glVertex*(...);  
glEnd();
```

The **type** parameter in *glBegin* determines how the vertices are combined to form geometric shapes.



# ► Drawing Pixels, Lines, and Polygons using OpenGL

## ❖ Creating Lines in OpenGL:

- In OpenGL, the term "line" refers to a line segment. There are ways to specify a connected series of line segments, or even a closed connected series of segments. These lines are specified in terms of the vertices at their endpoints.
- With OpenGL, we can specify lines with **different widths** and lines that are stippled in various ways, dotted, dashed, and so on.
- Drawing a line with OpenGL is straightforward using the following code:

```
glBegin(GL_LINES);  
    glVertex2f(x1, y1);  
    glVertex2f(x2, y2);  
glEnd();
```

# ► Drawing Pixels, Lines, and Polygons using OpenGL

## ❖ Creating Polygons in OpenGL:

- A polygon has two sides, **the front and the back**, and might be rendered differently depending on which side is facing the viewer.
- This allows us to have cutaway views of solid objects in which there is an obvious distinction between the parts that are inside and those that are outside.
- **By default**, both front and back faces are drawn in the same way; **to change this**, or to draw only outlines or vertices, use the function:

**`glPolygonMode();`**

# ► Drawing Pixels, Lines, and Polygons using OpenGL

## ◆ Creating Polygons in OpenGL:

- We can draw polygon using following code segment:

```
glBegin(GL_POLYGON); // Draw a Quad
glVertex3f(-0.5f, 1.0f, 0.0f); // Top
glVertex3f(-1.0f, 0.0f, 0.0f); // Left
glVertex3f(-0.5f, -1.0f, 0.0f); // Bottom left
glVertex3f(0.5f, -1.0f, 0.0f); // Bottom right
glVertex3f(1.0f, 0.0f, 0.0f); // Top right
glVertex3f(0.5f, 1.0f, 0.0f); // Bottom Right
glEnd();
```

## ► Basic Lighting:

- ❖ Lighting in the real world is **extremely complicated** and depends on **many factors**.
- ❖ Lighting in OpenGL is based on approximations of **reality using simplified models** that are much **easier to process** and look relatively similar.
- ❖ These lighting models are based on the **physics of light** as we understand it. One of these models is called the **Phong lighting model**.
- ❖ The major building blocks of the Phong model consist of the following **three components**:
  - Ambient Lighting
  - Diffused Lighting
  - Specular Lighting

# ► Types of Lighting:

## ❖ Ambient Lighting:

- **Even when it is dark**, there is usually still some light somewhere in the world (the moon, a distant light) so objects are **almost never completely dark**.
- **To simulate this**, we use an ambient lighting constant that always gives the object some color.
- Ambient illumination is light that's been scattered so much by the environment that its direction is **impossible to determine**. It seems to come from all directions.

## ❖ Diffuse Lighting:

- This is the **most visually significant** component of the lighting model.
- The more a part of an object faces the light source, the brighter it becomes.
- The diffuse component of the light that comes from one direction is the brightest if it comes squarely down on a surface than if it barely glances off the surface. Once it hits a surface it is **scattered equally in all directions**, so it appears equally bright.

# ► Types of Lighting:

## ◆ Specular Lighting:

- It simulates the **bright spot of a light** that appears on shiny objects.
- Specular highlights are often more inclined to the color of the light than the color of the object.
- Specular light comes from a particular direction, and it tends to bounce off the surface in a preferred direction.
- Shiny metal or plastic has a high specular component.

# ► Assignment-10:

Deadline: **14<sup>th</sup> March, 2025**

- ❖ Give some basic color model. Give the basic command to draw the pixel and polygon in OpenGL.
- ❖ Mention any two color command in OpenGL.
- ❖ Short Notes:
  - OpenGL
  - OpenGL Header Files
  - OpenGL Library Files
- ❖ Give the advantages of OpenGL over Microsoft's Direct3D.
- ❖ OpenGL is written in which language? Is it possible to implement (or use) same library in programming language other than that?
- ❖ What does Rasterization means? How is it different from vector graphics?
- ❖ What is the difference between `glColor3f()` & `glClearColor()`?
- ❖ Is OpenGL platform independent? Explain



▶ **THANKS!**

**Do you have any questions?**

**[theciceerguy@p\\_m.me](mailto:theciceerguy@p_m.me)**

**9841893035**

**[ghimires.com.np](http://ghimires.com.np)**